

ARK-458 — Preregistration

Cloud IAM Role Grant · Exact-Action Binding

Series: ExecutionProof authorization-boundary corpus — **Production-Boundary phase (P01)**

Experiment ID: ARK-458

Institution: Remnant Fieldworks Inc.

PI: Derek Adam Hone

Substrate: Classical software (no quantum hardware, no cryptographic security claims)

Guards: Dual independent — V1 (JavaScript), V2 (Python)

Preregistration status: LOCKED before official execution (see `MANIFEST.txt`)

Commercial / standards / IP justification (required by RF covenant)

- **Commercial purpose:** Directly answers **Prospect Question #3** — “Can a changed action be rejected after it was approved?” — in a **recognizable enterprise context** (an AWS-style IAM role grant). This is the first ExecutionProof **production-boundary integration**, moving beyond abstract authorization tuples to a concrete, buyer-legible action: granting an IAM role.
 - **RF-100 requirement:** Exercises the **exact-action-binding** conformance requirement — that an approval authorizes one specific action and nothing broader.
 - **Patent-family connection:** Produces dated **working-example** evidence for the exact-action / no-privilege-drift mechanisms described in the filed parent application `19/529,283` and the AI-governance / proof CIP family. Working-example evidence only — it does not legally validate any claim, and adds no new matter to any filed application.
-

Question

When an authorization is APPROVED bound to a specific IAM role-grant action (an exact action tuple), can that approval be used to execute a **mutated** grant — a higher-privilege role, a different grantee, a different account, a broadened permission set, a stripped scope condition, multi-dimension tampering, or a superficial “confusable” near-match? A correct guard must **ALLOW only on an exact action-tuple match** and **DENY every mutation**.

Binding dimensions (the “IAM action tuple”)

Dimension	Role in the grant
<code>principal</code>	ARN of the grantee (who receives the role)
<code>role</code>	the role / managed policy being granted
<code>account</code>	target AWS account id
<code>permission_set</code>	canonical hash of the effective permission scope (policy body)
<code>condition</code>	scope condition (region / MFA / resource constraint)

Design

- **8 arms × 100 scenarios = 800 evaluation decisions.**
- Each scenario: one approved action (`authorization.binding`) + one execution action (`execution.action`).
- Arm 1 is an exact copy (expect ALLOW). Arms 2–8 mutate the action (expect DENY).

Arm	Condition	Expected
1	BASELINE-exact-action-match	ALLOW
2	ROLE-ESCALATION (strictly higher-privilege role)	DENY
3	PRINCIPAL-SWAP (grantee changed)	DENY
4	ACCOUNT-SWAP (target account changed)	DENY
5	PERMISSION-EXPANSION (same role name, broadened policy body)	DENY
6	CONDITION-STRIP (scope condition removed / loosened)	DENY
7	MULTI-DIM-TAMPER (2–3 dimensions mutated at once)	DENY
8	CONFUSABLE-NEAR-MATCH (trailing space / case-flip / zero-width / homoglyph)	DENY

Guard rule (both V1 and V2, independently implemented)

ALLOW **iff** the execution action equals the approved action by **code-point-exact string equality on every binding dimension**. No normalization, no case folding, no whitespace trimming, no homoglyph mapping, and **no privilege “superset” reasoning** (a broader role is still a different action). Any mismatch → DENY (fail-closed).

Metrics

- `g_allow_exact` = ALLOW rate on arm 1.
- `g_deny_mutation_min` = minimum DENY rate across arms 2-8.
- `g_margin` = $\min(\text{g_allow_exact}, \text{g_deny_mutation_min}) - 0.05$.
- Secondary: V1↔V2 decision concordance (target 100%).

Pass criteria (fixed before execution)

- **C1:** `g_allow_exact` ≥ 0.95
- **C2:** `g_deny_mutation_min` ≥ 0.95
- **C3:** `g_margin` ≥ 0.90
- **Verdict** = **PASS** iff $C1 \wedge C2 \wedge C3$, else FAIL. Result is published either way.

Kill conditions / effectiveness gate

1. **Effectiveness oracle:** every attack-arm scenario must genuinely differ from the approved action; the baseline must be a true exact match; arm 2 must be a strict escalation; arm 8 must be confusable under a lenient normalizer. Any violation ⇒ ABORT (no verdict).
2. **Kill-gate (negative control):** a deliberately lenient comparator must be shown to wrongly ALLOW a detectable number of mutations — proving the testbed is falsifiable. If nothing can fail the test, the test is void.

Publication rule

The outcome (PASS, FAIL, or ABORT) will be published to GitHub with raw per-arm results, and defensively published to Zenodo (CC BY 4.0), regardless of verdict.

Honest bounds

- Classical software test of **authorization control logic**, not a test of AWS, not a cryptographic security proof, not a production-readiness certification.
- `permission_set` is a plain SHA-256 stand-in for “the exact policy document”; it is an identity token for the effective scope, not a security primitive.
- Results are bounded to this scenario model, these arms, and this seed family.
- No claim that this experiment legally validates any patent claim or certifies RF-100 conformance.

Preregistered under the Remnant Fieldworks Standing Covenant (preregister → lock → execute → publish all outcomes). To God be the glory. Proof Before Power. Verification Before Execution.